0621. Task Scheduler

Medium · Heap / Priority Queue

leetcode.com/problems/task-scheduler

Problem

- Tasks are letters; identical tasks need at least n intervals between them. Each interval runs one task or idles. Return the minimum total intervals.
- Example: [A,A,A,B,B,B], n = 2 → 8 (A B idle A B idle A B).

Intuition

- The most frequent task dictates the skeleton: maxFreq blocks, separated by n slots: (maxFreq - 1) (n + 1) + (number of tasks with maxFreq).
- If there are enough other tasks to fill every gap, no idling is needed and the answer is just the task count.

Brute Force

- Simulate with a max-heap of counts and a cooldown queue: O(total log 26). Correct, and good to show.

Optimal Approach

- Count frequencies; maxFreq and countMax = how many letters reach maxFreq.
- Answer = max(tasks.length, (maxFreq - 1) (n + 1) + countMax).

Worked Example

```
[A,A,A,B,B,B], n = 2.

maxFreq=3 (A and B) countMax=2
(3-1)*(2+1)+2 = 8 ; tasks=6 ; answer max(6,8)=8
layout: A B _ | A B _ | A B
```

Java Solution

```java
public class TaskScheduler {
    public int leastInterval(char[] tasks, int n) {
        int[] f = new int[26];
        for (char t : tasks) f[t - 'A']++;
        int max = 0, countMax = 0;
        for (int x : f) {
            if (x > max) { max = x; countMax = 1; }
            else if (x == max) countMax++;
        }
        return Math.max(tasks.length, (max - 1) * (n + 1) + countMax);
    }

    public int leastIntervalSim(char[] tasks, int n) {
        int[] f = new int[26];
        for (char t : tasks) f[t - 'A']++;
        PriorityQueue<Integer> pq = new PriorityQueue<>(Collections.reverseOrder());
        for (int x : f) if (x > 0) pq.add(x);
        Deque<int[]> cool = new ArrayDeque<>();
        int time = 0;
        while (!pq.isEmpty() || !cool.isEmpty()) {
            time++;
            if (!pq.isEmpty()) {
                int left = pq.poll() - 1;
                if (left > 0) cool.add(new int[]{left, time + n});
            }
            if (!cool.isEmpty() && cool.peek()[1] == time) pq.add(cool.poll()[0]);
        }
        return time;
    }
}
```

Complexity

- Time: O(N). Space: O(26).

Pitfalls

- Forgetting the max with tasks.length when n is small (n = 0 → answer is N).